

## Most interesting design decision

I optimized for a real walkthrough, not a screenshot. An agent stands in an empty house and types quantities fast, so the running total had to update with *zero* jank. Rather than re-render the whole screen on every keystroke (which loses input focus and stutters), I built a two-path render: structural changes (checking an item, expanding a group, switching rooms) do a simple full re-render, but quantity entry *patches only the affected DOM nodes* — the line total, its group total, the header total, and the progress bar. The result feels native while the code stays simple. The same “field-first” instinct led me to drop the reference app’s Tailwind play-CDN and hand-write the CSS, so the app paints instantly and is genuinely offline (no 3 MB runtime compiler to cache). Internally, everything keys off the price-list `id` and a `room::item` state model, so the CSV stays the single source of truth and rooms can be cloned freely.

## What is broken or fragile

- **Serial OCR.** The age decoder is an honest best-effort heuristic across common HVAC / appliance serial formats (explicit year, Carrier week-year, Rheem year-week, 2-digit prefixes), not an exhaustive per-brand database — so it always lets the agent confirm or override the detected year. Tesseract also needs one online load before it can run offline.
- **Photo storage.** Photos live in `localStorage` (downscaled to 1280px). A very photo-heavy project can approach the browser quota; the app degrades by keeping selections and dropping the heaviest photo data rather than failing the save. A production build would move photos to IndexedDB.
- **Local-only.** State is per-device. There is no multi-agent sync yet.

## Creative additions (and why)

My required creative addition — a feature not listed in the brief — is the **Deal Analyzer**. I also extended the required photo capture into a serial age decoder. Both target the part of the job the brief says hurts most: missed hidden costs and mispriced offers.

- **Deal Analyzer.** The live repair total flows into a Maximum Allowable Offer ( $MAO = ARV \times (1 - \text{margin}) - \text{repairs} - \text{costs}$ ), projected profit, and a GO / TIGHT / NO verdict. It connects every checkbox in the walkthrough to the only number that decides the deal, and rides along into the Excel export. I chose it because the brief frames the whole business around ARV minus repairs minus offer — this makes that math live.
- **Serial-photo OCR + equipment age decoder.** Scan an HVAC / water-heater / appliance nameplate; the app reads the serial, decodes the manufacture year, and flags units past their service life — catching the missed furnace or water heater that swings an estimate by thousands.

## What I would ship next (two more days)

- Cloud sync so a team shares projects across devices, with offline-first conflict handling.
- A materials / scope-of-work export grouped by trade, generated from the same selections.
- ARV assist: pull a rough comp-based ARV so the Deal Analyzer needs less manual input.
- Harden OCR with a per-brand decode library and an on-device confidence score.

## Role of AI tools

I built this with Claude Code (Opus) as a pair programmer. I owned the architecture, the data model, the two creative features, the offer math, and all product decisions; AI accelerated scaffolding the 107-item data set from the CSV, iterating on the mobile UI, and debugging the export and service-worker caching. Every line was reviewed by me and the full flow was verified in-browser (live totals, ZIP/Excel export, OCR, offline) before shipping.